

TOUGE RACER

Racing Game Template

Documentation

By SainDeveloper

Overview

Touge Racer is a powerful and easy-to-use arcade racing game template for Unity. This template provides all the core systems needed to create a touge/mountain pass-style racing game, allowing you to focus on content development and customization.

Key Features

- **Arcade Vehicle Physics:** Responsive and fun car physics with an automatic drift system.
- **Smooth & Smart AI:** Enemy AI that navigates the track smoothly using a spline system.
- **Minimap System:** Built-in minimap to help players track their position and the course.
- **Complete Race Management:** Ready-to-use race management system (countdown, laps, finish line, leaderboard).
- **Dynamic Camera:** Multi-angle camera system using Cinemachine.
- **Organized Structure:** Well-structured code and assets, making it easy to learn and customize.

Compatibility

Unity Version: Compatible with Unity 6000.0.37f1 LTS and newer versions.

Dependencies: This template requires the following Unity packages:

- Input System
- TextMeshPro
- Cinemachine
- Splines
- Shader Graphs

(Please ensure these packages are imported into your project first if they are not already present)

First Step

After the import process is complete, open the main scene located at `Assets/Touge Racer/Scenes/MainMenu.unity`. Run this scene to get started.

Project Settings

The template requires specific Layers to function correctly.

1. Go to Edit -> Project Settings -> Tags and Layers.
2. Add two new layers named "**Car**" and "**GhostCar**".
3. **IMPORTANT:** Go to Edit -> Project Settings -> Physics.
4. In the "Layer Collision Matrix" column, **UNCHECK** the intersection between the Car and GhostCar layers. This prevents AI cars and the player's car from physically colliding.

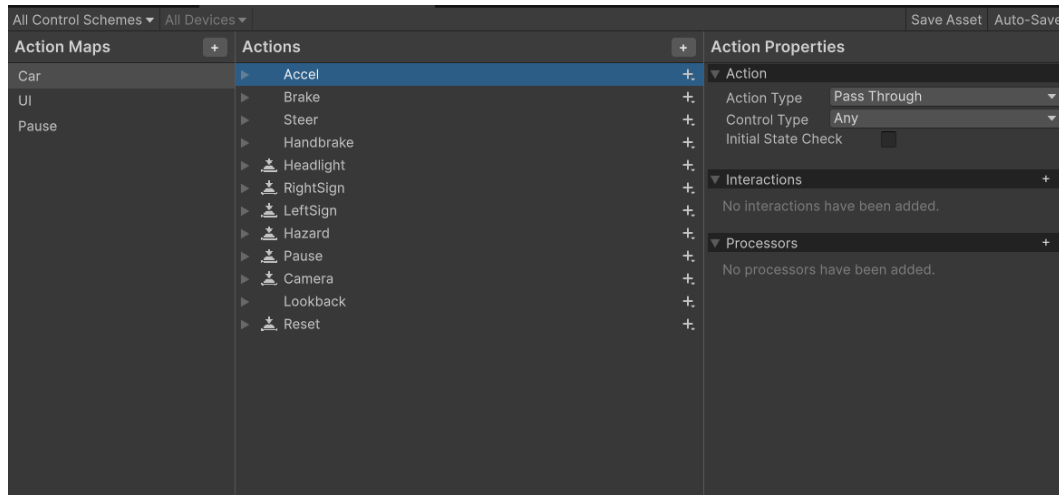
Shader Graphs

There is a shader graphs in Materials folder that called **Dither Shader Graph**. It used for car ghostmode. You can use your own shader but can't use for ghostmode visual.



Controls

The car controls use Unity's Input System. Input mapping can be viewed and customized in **Assets/Touge Racer/Scripts/Input/**.



Create new Car

You can create new car by duplicate prefabs that already exists in **Assets/Touge Racer/Prefabs/Cars**.

Child objects

- **AccelPoint** : Point to apply force.
- **Raypoints** : Point for suspension system.
- **Tires** : Contain tire meshes to apply tire rotation effect.
- **Skidmarks** : Contain trail renderer to apply skidmark effect.
- **Smokes** : Smoke effect not used particle system.
- **Visuals** : This is where you can change the car.

After finish create new car, make prefab from what has been made and add your new car to **CarDatabase** in Resources folder.

If your car spinning strangely, it happened because the angular damping of rigidbody was too low. Change it to higher value to fix it (for example 10).

Create new Stage

Required object that must be present in Scene:

- **Race track model**
- **Starting Points**: To determine the starting position

- **Spline:** To determine racing route
- **FinishLine:** Can be found in **Prefabs** folder
- **RaceGUI:** Can be found in **Prefabs** folder
- **MinimapManager:** Can be found in **Prefabs>GameManager** folder
- **CameraManager:** Can be found in **Prefabs>GameManager** folder
- **RaceManager:** Assign all object to **RaceManager**

After finish setting scene, create new **StageData** by **Create>Touge Racer>Stage Data**. After that assign new stage data to **StagesDatabase** in **Resources** folder.

Minimap System

How to setup minimap to scene:

1. Assign **RaceGUI** in **Prefabs** folder to the scene and change map source image in **MinimapMask>MinimapImage**.
2. Assign **MinimapManager** in **Prefabs>GameManager** folder to the scene and assign **MinimapImage** to **MinimapManager**.
3. Adjust **MinimapManager** Transform until fit to your map.
4. Now minimap can be use.

Scripts

Core Scripts

CarController: Core of the car physics (speed, drift, etc.).

CarInput: Handles input, both from the player (User) and AI.

CarAudioSystem: Controls all car-related sounds.

RaceManager: The brain of the race. Manages countdown, laps, positions, and finish.

CameraManager: Controls camera switching and behavior using Cinemachine.

MinimapManager: Manages minimap rendering and display.

RaceEvents: Static class containing all event systems.

Scriptable Objects (Data)

CarDatabase: Collection of data for all cars available in the game.

StageData: Contains data specific to a track (name, scene, preview image, etc.).

StageDatabase: Collection of all StageData.

AIData: Contains parameters to control AI behavior and difficulty.

AIPool: A pool of AIData to be used by different AI cars.